Import Packages

In [ ]:
```
import requests
import pandas as pd
import numpy as np
import datetime

# Setting this option will print all collumns of a dataframe
pd.set_option('display.max_columns', None)
# Setting this option will print all of the data in a feature
pd.set_option('display.max_colwidth', None)
```

Define Functions

In [ ]:
```
# Takes the dataset and uses the rocket column to call the API and append the data to the list
# We want to see the name of the booster that is used for the rocket
def getBoosterVersion(data):
    for x in data['rocket']:
        if x:
            response = requests.get("https://api.spacexdata.com/v4/rockets/"+str(x)).json()
            BoosterVersion.append(response['name'])
```

In [ ]:
```
# Takes the dataset and uses the launchpad column to call the API and append the data to the list
# Here we also want to know the launch site name, its latitude and longitude.
def getLaunchSite(data):
    for x in data['launchpad']:
        if x:
         response = requests.get("https://api.spacexdata.com/v4/launchpads/"+str(x)).json()
         Longitude.append(response['longitude'])
         Latitude.append(response['latitude'])
         LaunchSite.append(response['name'])
```

In [ ]:
```
# Takes the dataset and uses the payloads column to call the API and append the data to the lists
# Here we want to see the payload mass and the specific orbit this payload is going to.
def getPayloadData(data):
    for load in data['payloads']:
       if load:
        response = requests.get("https://api.spacexdata.com/v4/payloads/"+load).json()
        PayloadMass.append(response['mass_kg'])
        Orbit.append(response['orbit'])
```

In [ ]:
```
# Takes the dataset and uses the cores column to call the API and append the data to the lists
''' With this function we want to see the  outcome of the landing, the type of the landing, number of flights with that core,
whether gridfins were used, whether the core is reused, wheter legs were used, the landing pad used, the block of the core which is a number used to seperate version of cores,
the number of times this specific core has been reused, and the serial of the core.

'''
def getCoreData(data):
    for core in data['cores']:
            if core['core'] != None:
                response = requests.get("https://api.spacexdata.com/v4/cores/"+core['core']).json()
                Block.append(response['block'])
                ReusedCount.append(response['reuse_count'])
                Serial.append(response['serial'])
            else:
                Block.append(None)
                ReusedCount.append(None)
                Serial.append(None)
            Outcome.append(str(core['landing_success'])+' '+str(core['landing_type']))
            Flights.append(core['flight'])
            GridFins.append(core['gridfins'])
            Reused.append(core['reused'])
            Legs.append(core['legs'])
            LandingPad.append(core['landpad'])
```

Grab API link to past SpaceX launches

In [ ]:
```
# This should be a dynamic link, so results will probably change over time.
spacex_url="https://api.spacexdata.com/v4/launches/past"
```

Create get request

In [ ]:
```
# The results of this requests should be in a JSON format.
response = requests.get(spacex_url)
```

Check Response Code

In [ ]:
```
# Code '200' should indicate successful retrieval
print(response.status_code)
```

```
200
```

Normalize Data for Dataframe Creation

In [ ]:
```
# Use json_normalize meethod to convert the json result into a dataframe
data = pd.json_normalize(response.json())

data.head(3)
```

Out[ ]:

	static_fire_date_utc	static_fire_date_unix	net	window	rocket	success	failures	details	crew	ships	capsules	payloads	launchpad	flight_number	
0	2006-03-17T00:00:00.000Z	1.142554e+09	False	0.0	5e9d0d95eda69955f709d1eb	False	[{'time': 33, 'altitude': None, 'reason': 'merlin engine failure'}]	Engine failure at 33 seconds and loss of vehicle	[]	[]	[]	[5eb0e4b5b6c3bb0006eeb1e1]	5e9e4502f5090995de566f86	1	Fa
1	NaN	NaN	False	0.0	5e9d0d95eda69955f709d1eb	False	[{'time': 301, 'altitude': 289, 'reason': 'harmonic oscillation leading to premature engine shutdown'}]	Successful first stage burn and transition to second stage, maximum altitude 289 km, Premature engine shutdown at T+7 min 30 s, Failed to reach orbit, Failed to recover first stage	[]	[]	[]	[5eb0e4b6b6c3bb0006eeb1e2]	5e9e4502f5090995de566f86	2	D
2	NaN	NaN	False	0.0	5e9d0d95eda69955f709d1eb	False	[{'time': 140, 'altitude': 35, 'reason': 'residual stage-1 thrust led to collision between stage 1 and stage 2'}]	Residual stage 1 thrust led to collision between stage 1 and stage 2	[]	[]	[]	[5eb0e4b6b6c3bb0006eeb1e3, 5eb0e4b6b6c3bb0006eeb1e4]	5e9e4502f5090995de566f86	3	Tra

A lot of the entries here are ID's.

- Rocket column is literally just ID numbers. This isn't very helpful.
- We should fetch the ID numbers and their corresponding rocket models. Specifically we will be using columns rocket, payloads, launchpad, and cores.

In [ ]:
```
# Lets take a subset of our dataframe keeping only the features we want and the flight number, and date_utc.
data = data[['rocket', 'payloads', 'launchpad', 'cores', 'flight_number', 'date_utc']]

# We will remove rows with multiple cores because those are falcon rockets with 2 extra rocket boosters and rows that have multiple payloads in a single rocket.
data = data[data['cores'].map(len)==1]
data = data[data['payloads'].map(len)==1]

# Since payloads and cores are lists of size 1 we will also extract the single value in the list and replace the feature.
data['cores'] = data['cores'].map(lambda x : x[0])
data['payloads'] = data['payloads'].map(lambda x : x[0])

# We also want to convert the date_utc to a datetime datatype and then extracting the date leaving the time
data['date'] = pd.to_datetime(data['date_utc']).dt.date

# Using the date we will restrict the dates of the launches
data = data[data['date'] <= datetime.date(2020, 11, 13)]
```

What do we want to know from our data?

1. From the rocket we would like to learn the booster name
2. From the payload we would like to learn the mass of the payload and the orbit that it is going to
3. From the launchpad we would like to know the name of the launch site being used, the longitude, and the latitude.
4. From the cores we would like to learn the outcome of the landing, the type of the landing, number of flights with that core, whether gridfins were used, whether the core is reused, whether legs were used, the landing pad used, the block of the core which is a number used to seperate version of cores, the number of times this specific core has been reused, and the serial of the core.

The data from these requests will be stored in lists and will be used to create a new dataframe.

In [ ]:
```
#Global variables for appending fetched values for the variables in our dataset that are just ID numbers.
BoosterVersion = []
PayloadMass = []
Orbit = []
LaunchSite = []
Outcome = []
Flights = []
GridFins = []
Reused = []
Legs = []
LandingPad = []
Block = []
ReusedCount = []
Serial = []
Longitude = []
Latitude = []
```

Fetch Model Numbers and Exact Names from Columns with Only IDs

In [ ]:
```
# Call getBoosterVersion
#  This should get booster version
getBoosterVersion(data)
```

Check Booster Names (This will have a very long compile time)

In [ ]:
```
BoosterVersion[0:5]
```

Out[ ]:
```
['Falcon 1', 'Falcon 1', 'Falcon 1', 'Falcon 1', 'Falcon 9']
```

In [ ]:
```
# Call getLaunchSite
getLaunchSite(data)

# Call getPayloadData
getPayloadData(data)

# Call getCoreData
getCoreData(data)
```

This data with all the proper ID names and can now be used to make a dataframe.

- We'll first make a dictionary from this.

In [ ]:
```
launch_dict = {'FlightNumber': list(data['flight_number']),
'Date': list(data['date']),
'BoosterVersion':BoosterVersion,
'PayloadMass':PayloadMass,
'Orbit':Orbit,
'LaunchSite':LaunchSite,
'Outcome':Outcome,
'Flights':Flights,
'GridFins':GridFins,
'Reused':Reused,
'Legs':Legs,
'LandingPad':LandingPad,
'Block':Block,
'ReusedCount':ReusedCount,
'Serial':Serial,
'Longitude': Longitude,
'Latitude': Latitude}
```

In [ ]:
```
# Create a data from launch_dict
df = pd.DataFrame(launch_dict)
df.head()
```

Out[ ]:

	FlightNumber	Date	BoosterVersion	PayloadMass	Orbit	LaunchSite	Outcome	Flights	GridFins	Reused	Legs	LandingPad	Block	ReusedCount	Serial	Longitude	Latitude
0	1	2006-03-24	Falcon 1	20.0	LEO	Kwajalein Atoll	None None	1	False	False	False	NaN	NaN	0	Merlin1A	167.743129	9.047721
1	2	2007-03-21	Falcon 1	NaN	LEO	Kwajalein Atoll	None None	1	False	False	False	NaN	NaN	0	Merlin2A	167.743129	9.047721
2	4	2008-09-28	Falcon 1	165.0	LEO	Kwajalein Atoll	None None	1	False	False	False	NaN	NaN	0	Merlin2C	167.743129	9.047721
3	5	2009-07-13	Falcon 1	200.0	LEO	Kwajalein Atoll	None None	1	False	False	False	NaN	NaN	0	Merlin3C	167.743129	9.047721
4	6	2010-06-04	Falcon 9	NaN	LEO	CCSFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0003	-80.577366	28.561857

The data fetched are for all launches, using Falcon 1 and 9 boosters. However, for now, we're only interested in looking at Falcon 9 launches instead.

In [ ]:
```
# Filter for Falcon 9 launches
data_falcon9 = df[df['BoosterVersion'] != 'Falcon 1']
```

In [ ]:
```
data_falcon9
```

Out[ ]:

	FlightNumber	Date	BoosterVersion	PayloadMass	Orbit	LaunchSite	Outcome	Flights	GridFins	Reused	Legs	LandingPad	Block	ReusedCount	Serial	Longitude	Latitude
4	6	2010-06-04	Falcon 9	NaN	LEO	CCSFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0003	-80.577366	28.561857
5	8	2012-05-22	Falcon 9	525.0	LEO	CCSFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0005	-80.577366	28.561857
6	10	2013-03-01	Falcon 9	677.0	ISS	CCSFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0007	-80.577366	28.561857
7	11	2013-09-29	Falcon 9	500.0	PO	VAFB SLC 4E	False Ocean	1	False	False	False	NaN	1.0	0	B1003	-120.610829	34.632093
8	12	2013-12-03	Falcon 9	3170.0	GTO	CCSFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B1004	-80.577366	28.561857
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
89	102	2020-09-03	Falcon 9	15600.0	VLEO	KSC LC 39A	True ASDS	2	True	True	True	5e9e3032383ecb6bb234e7ca	5.0	12	B1060	-80.603956	28.608058
90	103	2020-10-06	Falcon 9	15600.0	VLEO	KSC LC 39A	True ASDS	3	True	True	True	5e9e3032383ecb6bb234e7ca	5.0	13	B1058	-80.603956	28.608058
91	104	2020-10-18	Falcon 9	15600.0	VLEO	KSC LC 39A	True ASDS	6	True	True	True	5e9e3032383ecb6bb234e7ca	5.0	12	B1051	-80.603956	28.608058
92	105	2020-10-24	Falcon 9	15600.0	VLEO	CCSFS SLC 40	True ASDS	3	True	True	True	5e9e3033383ecbb9e534e7cc	5.0	12	B1060	-80.577366	28.561857
93	106	2020-11-05	Falcon 9	3681.0	MEO	CCSFS SLC 40	True ASDS	1	True	False	True	5e9e3032383ecb6bb234e7ca	5.0	8	B1062	-80.577366	28.561857

90 rows × 17 columns

In [ ]:
```
data_falcon9.iloc[0:5, 0]
```

Out[ ]:
```
4     6
5     8
6    10
7    11
8    12
Name: FlightNumber, dtype: int64
```

In [ ]:
```
# All the flight numbers and indexes are out of order now that we've filtered out all Falcon 1 launches.
# Reset the flight numbers.
# Have every row in column 'FlightNumber' be set to a list ranging the length of the rows in the dataframe. This should be 90 from 90 entries and 17 features.
data_falcon9.iloc[:, 0] = list(range(1, data_falcon9.shape[0]+1))
data_falcon9.head()
```

Out[ ]:

	FlightNumber	Date	BoosterVersion	PayloadMass	Orbit	LaunchSite	Outcome	Flights	GridFins	Reused	Legs	LandingPad	Block	ReusedCount	Serial	Longitude	Latitude
4	1	2010-06-04	Falcon 9	NaN	LEO	CCSFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0003	-80.577366	28.561857
5	2	2012-05-22	Falcon 9	525.0	LEO	CCSFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0005	-80.577366	28.561857
6	3	2013-03-01	Falcon 9	677.0	ISS	CCSFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B0007	-80.577366	28.561857
7	4	2013-09-29	Falcon 9	500.0	PO	VAFB SLC 4E	False Ocean	1	False	False	False	NaN	1.0	0	B1003	-120.610829	34.632093
8	5	2013-12-03	Falcon 9	3170.0	GTO	CCSFS SLC 40	None None	1	False	False	False	NaN	1.0	0	B1004	-80.577366	28.561857

Check for Null or Missing Values

In [ ]:
```
data_falcon9.isnull().sum()
```

Out[ ]:
```
FlightNumber       0
Date               0
BoosterVersion     0
PayloadMass        5
Orbit              0
LaunchSite         0
Outcome            0
Flights            0
GridFins           0
Reused             0
Legs               0
LandingPad        26
Block              0
ReusedCount        0
Serial             0
Longitude          0
Latitude           0
dtype: int64
```

There are 5 missing values for the Payload mass and 26 for the landing pad.

- We can probably just impute the missing payload masses with something like a median/mean
- Not much need to anything much more complicated than that like KNN.

In [ ]:
```
# Find mean payload mass, and use that value to replace missing or NaN values in the payload mass column.
meanpayloadmass = data_falcon9['PayloadMass'].mean()
data_falcon9['PayloadMass'].replace(np.nan, meanpayloadmass, inplace = True)
```

```
/var/folders/nv/brc1sfvj70n58vqs4_3_5m8h0000gn/T/ipykernel_16045/2430582887.py:3: ChainedAssignmentError: A value is being set on a copy of a DataFrame or Series through chained assignment using an inplace method.
Such inplace method never works to update the original DataFrame or Series, because the intermediate object on which we are setting values always behaves as a copy (due to Copy-on-Write).

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' instead, to perform the operation inplace on the original object, or try to avoid an inplace operation using 'df[col] = df[col].method(value)'.

See the documentation for a more detailed explanation: https://pandas.pydata.org/pandas-docs/stable/user_guide/copy_on_write.html
  data_falcon9['PayloadMass'].replace(np.nan, meanpayloadmass, inplace = True)
```

Out[ ]:
```
4      6123.547647
5       525.000000
6       677.000000
7       500.000000
8      3170.000000
          ...
89    15600.000000
90    15600.000000
91    15600.000000
92    15600.000000
93     3681.000000
Name: PayloadMass, Length: 90, dtype: float64
```

In [ ]:
```
# Check for missing values again
# Payload Mass should have 0 missing values.
data_falcon9.isnull().sum()
```

Out[ ]:
```
FlightNumber       0
Date               0
BoosterVersion     0
PayloadMass        5
Orbit              0
LaunchSite         0
Outcome            0
Flights            0
GridFins           0
Reused             0
Legs               0
LandingPad        26
Block              0
ReusedCount        0
Serial             0
Longitude          0
Latitude           0
dtype: int64
```

Export API Fetched Data to CSV

In [ ]:
```
# data_falcon9.to_csv('Falcon9_api_data.csv', index = False)
```

Scrape for Data from Wikipedia

We're aiming to collect Falcon 9 historical launch records from a Wikipedia page titled List of Falcon 9 and Falcon Heavy launches. This page also has a decent amount of information on these launches.

https://en.wikipedia.org/wiki/List_of_Falcon_9_and_Falcon_Heavy_launches

Looking at the wiki page, it seems the data are stored in tables.

We can use beautifulsoup to parse and extract data from these tables.

In [ ]:
```
# Import packages needed to extract tabulated data.
from bs4 import BeautifulSoup
import re
import unicodedata
```

Define some functions that will help in extracting table data

In [ ]:
```
def date_time(table_cells):
    """
    This function returns the data and time from the HTML  table cell
    Input: the  element of a table data cell extracts extra row
    """
    return [data_time.strip() for data_time in list(table_cells.strings)][0:2]

def booster_version(table_cells):
    """
    This function returns the booster version from the HTML  table cell
    Input: the  element of a table data cell extracts extra row
    """
    out=''.join([booster_version for i,booster_version in enumerate( table_cells.strings) if i%2==0][0:-1])
    return out

def landing_status(table_cells):
    """
    This function returns the landing status from the HTML table cell
    Input: the  element of a table data cell extracts extra row
    """
    out=[i for i in table_cells.strings][0]
    return out


def get_mass(table_cells):
    mass=unicodedata.normalize("NFKD", table_cells.text).strip()
    if mass:
        mass.find("kg")
        new_mass=mass[0:mass.find("kg")+2]
    else:
        new_mass=0
    return new_mass


def extract_column_from_header(row):
    """
    This function returns the landing status from the HTML table cell
    Input: the  element of a table data cell extracts extra row
    """
    if (row.br):
        row.br.extract()
    if row.a:
        row.a.extract()
    if row.sup:
        row.sup.extract()

    column_name = ' '.join(row.contents)

    # Filter the digit and empty names
    if not(column_name.strip().isdigit()):
        column_name = column_name.strip()
        return column_name
```

Create url and get request

In [ ]:
```
wiki_url = "https://en.wikipedia.org/w/index.php?title=List_of_Falcon_9_and_Falcon_Heavy_launches"

# We'll need these headers to make the get request script to be seen as actual web browser
# Wikipedia and other sites will probably block this if we don't include the headers since it'll be see as bot traffic.

headers = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
                  "AppleWebKit/537.36 (KHTML, like Gecko) "
                  "Chrome/91.0.4472.124 Safari/537.36"
}

# use requests.get() method with the provided wiki_url and headers
# assign the response to a object
response = requests.get(static_wiki_url, headers = headers)

# Use BeautifulSoup() to create a BeautifulSoup object from a response text content
soup = BeautifulSoup(response.text, 'html.parser')

# Print title of HTML response to check if our request worked.
# If it works, this should be the html content of the entire wiki page.
print(soup.title)
```

```
<title>List of Falcon 9 and Falcon Heavy launches - Wikipedia</title>
```

Extract all columns/variable names from anything on this page that is in a table.

In [ ]:
```
# adjust code if needed for the older, static page
html_tables = soup.find_all('tr')

# Scanning the page, it looks like the launch data is on the third table in the page.
first_launch_table = html_tables[2]
print(first_launch_table)
```

```
<tr>
<th scope="col">Flight No.
</th>
<th scope="col">Date and<br/>time (<a href="/wiki/Coordinated_Universal_Time" title="Coordinated Universal Time">UTC</a>)
</th>
<th scope="col"><a href="/wiki/List_of_Falcon_9_first-stage_boosters" title="List of Falcon 9 first-stage boosters">Version,<br/>Booster</a> <sup class="reference" id="cite_ref-booster_11-0"><a href
="#cite_note-booster-11"><span class="cite-bracket">[</span>b<span class="cite-bracket">]</span></a></sup>
</th>
<th scope="col">Launch site
</th>
<th scope="col">Payload<sup class="reference" id="cite_ref-Dragon_12-0"><a href="#cite_note-Dragon-12"><span class="cite-bracket">[</span>c<span class="cite-bracket">]</span></a></sup>
</th>
<th scope="col">Payload mass
</th>
<th scope="col">Orbit
</th>
<th scope="col">Customer
</th>
<th scope="col">Launch<br/>outcome
</th>
<th scope="col"><a href="/wiki/Falcon_9_first-stage_landing_tests" title="Falcon 9 first-stage landing tests">Booster<br/>landing</a>
</th></tr>
```

In [ ]:
```
extract_column_from_header(first_launch_table.find_all('th')[0])
```

Out[ ]:
```
'Flight No.'
```

Extract Columns

In [ ]:
```
column_names = []

# Apply find_all() function with `th` element on first_launch_table
# Iterate each th element and apply the provided extract_column_from_header() to get a column name
# Append the Non-empty column name (`if name is not None and len(name) > 0`) into a list called column_names
for x in first_launch_table.find_all('th'):
    if extract_column_from_header(x) != None and len(extract_column_from_header(x)) > 0:
        column_names.append(extract_column_from_header(x))
    else:
        pass
```

Check column names

In [ ]:
```
print(column_names)
```

```
['Flight No.', 'Date and time ( )', 'Launch site', 'Payload', 'Payload mass', 'Orbit', 'Customer', 'Launch outcome']
```

Create a data frame by parsing the launch HTML tables

In [ ]:
```
launch_dict= dict.fromkeys(column_names)

# Remove irrelevant column
del launch_dict['Date and time ( )']

# Let's initial the launch_dict with each value to be an empty list
launch_dict['Flight No.'] = []
launch_dict['Launch site'] = []
launch_dict['Payload'] = []
launch_dict['Payload mass'] = []
launch_dict['Orbit'] = []
launch_dict['Customer'] = []
launch_dict['Launch outcome'] = []
# Add some new columns
launch_dict['Version Booster']=[]
launch_dict['Booster landing']=[]
launch_dict['Date']=[]
launch_dict['Time']=[]
```

Usually, HTML tables in Wiki pages are likely to contain unexpected annotations and other types of noises, such as reference links B0004.1[8], missing values N/A [e], inconsistent formatting, etc.

In [ ]:
```
extracted_row = 0

#Extract each table
for table_number,table in enumerate(soup.find_all('table',"wikitable plainrowheaders collapsible")):
   # get table row
    for rows in table.find_all("tr"):
        #check to see if first table heading is as number corresponding to launch a number
        if rows.th:
            if rows.th.string:
                flight_number=rows.th.string.strip()
                flag=flight_number.isdigit()
        else:
            flag=False
        #get table element
        row=rows.find_all('td')
        #if it is number save cells in a dictonary
        if flag:
            extracted_row += 1
            # Flight Number value
            # TODO: Append the flight_number into launch_dict with key `Flight No.`
            launch_dict['Flight No.'].append(flight_number)
            #print(flight_number)
            datatimelist=date_time(row[0])

            # Date value
            # TODO: Append the date into launch_dict with key `Date`
            date = datatimelist[0].strip(',')
            launch_dict['Date'].append(date)
            #print(date)

            # Time value
            # TODO: Append the time into launch_dict with key `Time`
            time = datatimelist[1]
            launch_dict['Time'].append(time)
            #print(time)

            # Booster version
            # TODO: Append the bv into launch_dict with key `Version Booster`
            bv=booster_version(row[1])
            if not(bv):
                bv=row[1].a.string

            launch_dict['Version Booster'].append(bv)
            print(bv)

            # Launch Site
            # TODO: Append the bv into launch_dict with key `Launch Site`
            launch_site = row[2].a.string
            launch_dict['Launch site'].append(launch_site)
            #print(launch_site)

            # Payload
            # TODO: Append the payload into launch_dict with key `Payload`
            payload = row[3].a.string
            launch_dict['Payload'].append(payload)
            #print(payload)

            # Payload Mass
            # TODO: Append the payload_mass into launch_dict with key `Payload mass`
            payload_mass = get_mass(row[4])
            launch_dict['Payload mass'].append(payload_mass)
            #print(payload)

            # Orbit
            # TODO: Append the orbit into launch_dict with key `Orbit`
            orbit = row[5].a.string
            launch_dict['Orbit'].append(orbit)
            #print(orbit)

            # Customer
            # TODO: Append the customer into launch_dict with key `Customer`
            customer = row[6].a.string
            launch_dict['Customer'].append(customer)
            #print(customer)

            # Launch outcome
            # TODO: Append the launch_outcome into launch_dict with key `Launch outcome`
            launch_outcome = list(row[7].strings)[0]
            launch_dict['Launch outcome'].append(launch_outcome)
            #print(launch_outcome)

            # Booster landing
            # TODO: Append the launch_outcome into launch_dict with key `Booster landing`
            booster_landing = landing_status(row[8])
            launch_dict['Booster landing'].append(booster_landing)
            #print(booster_landing)
```

```
F9 v1.07B0003.18
F9 v1.07B0004.18
F9 v1.07B0005.18
F9 v1.07B0006.18
F9 v1.07B0007.18
F9 v1.17B10038
F9 v1.1
F9 v1.1
F9 v1.1
F9 v1.1
F9 v1.1
F9 v1.1[
F9 v1.1[
F9 v1.1[
F9 v1.1[
F9 v1.1[
F9 v1.1[
F9 v1.1[
F9 v1.1[
F9 FT[
F9 v1.1[
F9 FT[
F9 FT[
F9 FT[
F9 FT[
F9 FT[
F9 FT[
F9 FT[
F9 FT[
F9 FT[
F9 FT[
F9 FT△[
F9 FT[
F9 FT[
F9 FT[
F9 FTB1029.2195
F9 FT[
F9 FT[
F9 B4[
F9 FT[
F9 B4[
F9 B4[
F9 FTB1031.2220
F9 B4[
F9 FTB1035.2227
F9 FTB1036.2227
F9 B4[
F9 FTB1032.2245
F9 FTB1038.2268
F9 B4[
F9 B4B1041.2268
F9 B4B1039.2292
F9 B4[
F9 B5311B1046.1268
F9 B4B1043.2322
F9 B4B1040.2268
F9 B4B1045.2336
F9 B5
F9 B5349B1048[
F9 B5B1046.2354
F9 B5[
F9 B5B1048.2364
F9 B5B1047.2268
F9 B5B1046.3268
F9 B5[
F9 B5[
F9 B5B1049.2397
F9 B5B1048.3399
F9 B5[]413
F9 B5[
F9 B5B1049.3434
F9 B5B1051.2420
F9 B5B1056.2465
F9 B5B1047.3472
F9 B5
F9 B5[
F9 B5B1056.3482
F9 B5
F9 B5
F9 B5
F9 B5
F9 B5
F9 B5
F9 B5
F9 B5[
F9 B5
F9 B5
F9 B5
F9 B5B1058.2544
F9 B5
F9 B5B1049.6544
F9 B5
F9 B5B1060.2563
F9 B5B1058.3565
F9 B5B1051.6568
F9 B5
F9 B5
F9 B5[
F9 B5
F9 B5 △[
F9 B5 △[
F9 B5 △
F9 B5 △
F9 B5
F9 B5B1051.8609
F9 B5B1058.5613
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[44], line 72
     68             #print(orbit)
     69 
     70             # Customer
     71             # TODO: Append the customer into launch_dict with key `Customer`
---> 72             customer = row[6].a.string
     73             launch_dict['Customer'].append(customer)
     74             #print(customer)
     75 

AttributeError: 'NoneType' object has no attribute 'string'
```

I think the error seen here should be fine. It should from the data having no customer listed.

In [ ]:
```
# Create Dataframe from parsed launch records
df= pd.DataFrame({key:pd.Series(value) for key, value in launch_dict.items()})
df.head()
```

Out[ ]:

	Flight No.	Launch site	Payload	Payload mass	Orbit	Customer	Launch outcome	Version Booster	Booster landing	Date	Time
0	1	CCAFS	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success\n	F9 v1.07B0003.18	Failure	4 June 2010	18:45
1	2	CCAFS	Dragon	0	LEO	NASA	Success	F9 v1.07B0004.18	Failure	8 December 2010	15:43
2	3	CCAFS	Dragon	525 kg	LEO	NASA	Success	F9 v1.07B0005.18	No attempt\n	22 May 2012	07:44
3	4	CCAFS	SpaceX CRS-1	4,700 kg	LEO	NASA	Success\n	F9 v1.07B0006.18	No attempt	8 October 2012	00:35
4	5	CCAFS	SpaceX CRS-2	4,877 kg	LEO	NASA	Success\n	F9 v1.07B0007.18	No attempt\n	1 March 2013	15:10

Export Scraped Data to CSV

In [ ]:
```
# df.to_csv('static_spacex_web_scraped.csv', index=False)
```